

CASINO : A New Monte Carlo Code in C Language for Electron Beam Interactions—Part II : Tabulated Values of the Mott Cross Section

DOMINIQUE DROUIN, PIERRE HOVINGTON, RAYNALD GAUVIN

Département de Génie Mécanique, Université de Sherbrooke, Sherbrooke, Québec, Canada

Summary: This paper presents routines to compute the Mott cross section used in the CASINO program (Monte Carlo Simulation of electron trajectory in solid). The routines used tabulated values of the Mott cross section computed in the work of Czyzewski *et al.* (1990). The cross section is available over the range 0.02 to 30 keV and for the first 94 elements of the Periodic Table. The routines are written in C language and use a binary file to interpolate the cross section. The first routine computes the total Mott cross sections; the second calculates the polar angle of collision. Backscattered coefficients computed using different cross section are compared for C, Al, Ag, and Au. The Rutherford (1911) cross section and the available empirical equations (Browning *et al.* 1994, Gauvin and Drouin 1993) are compared to tabulated values of Mott. Also, the energy distribution of backscattered electrons is shown for Al and Au at 10 keV. Finally, the relative computation times for the different Mott cross sections are compared. It was found that tabulated Mott cross sections are more accurate and faster than any empirical Mott cross sections. The tabulated Mott cross sections are even faster than simple Rutherford cross sections.

Key words: Mott elastic cross section, Monte Carlo simulation, backscattered electrons, experimental stopping power

1. Introduction

During the past few years, much attention has been directed toward low energy modelization of electron interaction with solids. This interest arises from the application of scanning electron microscopy (SEM) to the analysis of non- or poor conductor materials such as polymers, ceramics, biological material, and composites. With a low-energy electron beam, that is, 1 keV, it is possible to observe such material without charging problems. This type of microscopy has been

enhanced by the advent of a new type of SEM called the field emission scanning electron microscope (FESEM). This apparatus is able to produce a low-energy electron beam with a diameter on the order of nanometers and a high-density probe current. Also, the low-beam energy produces a small interaction volume that can be used for high-resolution microanalysis of heterogeneous samples.

Monte Carlo programs are widely used to support microscope users. To achieve accurate results, the physics behind the Monte Carlo simulation must be appropriate for all ranges of energy used. For any standard Monte Carlo program, two phenomena must be modeled: elastic collision and energy loss. In the diffusion process, inelastic collisions are usually neglected because of the low angle resulting from such events. Instead, all inelastic event are considered to be uniformly distributed along the paths of the electrons. Thus, a continuous slowing-down equation is used to model this phenomenon. Stopping power based on the Bethe expression (Bethe 1933) is usually used to model the energy loss. For low energy, this expression must be reevaluated since the number of inelastic events is decreasing. For further information, interested readers are referred to the work of Hovington *et al.* (1997a). For elastic events, the Rutherford theory (Rutherford 1911) has been widely used in the first Monte Carlo programs. The popularity of this model is justified by its simplicity of use and its short computation time. This model is accurate for high-energy interaction (> 10 keV) because it is based on the first Born approximation. However, the Rutherford theory is not valid for low energy. At this energy range, the more accurate Mott theory (Mott and Massey 1949) is preferable. Empirical forms of the Mott cross section have been developed recently (Browning *et al.* 1994, Drouin *et al.* 1994, Gauvin and Drouin 1993). All these models are adapted from the Rutherford cross section by adding extra terms to the equations. In this paper it is shown that interpolation from the Mott cross section data base is, by definition, more accurate than the analytical fit, needs less computation time than the Rutherford cross section, and thus it is also faster than empirical forms of the Mott cross section.

2. Mott Cross Section for Monte Carlo Simulation

The differential Mott cross sections for each element of the periodic table were computed and published in the work of

Address for reprints

Dominique Drouin
Département de Génie Mécanique
Université de Sherbrooke
2500 Boulevard Université
Sherbrooke, Québec
Canada J1K 2R1

Czyzewski *et al.* (1990). The Dirac equation reduced to a first-order differential equation was solved numerically to determine the Mott elastic scattering cross section. Three different atomic potentials were used to determine the Mott cross section: the Thomas-Fermi-Dirac (TFD) (Bonham and Strand 1963), the Hartree-Fock (HF) (Strand and Bonham 1964), and the relativistic Hartree-Fock-Slater (HFS) (Cox and Bonham 1967). These potentials lead to two distinct data sets that are summarized in Table I. Each data set is associated with a letter and the program will refer to this letter for the choice of a potential. Figure 1 shows the differential scattering cross section of Si at 0.1 keV for the two types of potential.

In the Monte Carlo simulation, the total elastic cross section is used to determine the distance between collisions. To compute the total cross section (σ_T), the differential cross section must be integrated using the following equation :

$$\sigma_T = 2\pi \int_0^\pi \frac{d\sigma}{d\Omega} \sin \theta d\theta \quad (1)$$

where θ is the scattering angle and $\frac{d\sigma}{d\Omega}$ is the differential

cross section. Using a numerical integration, the total cross section is computed for 26 different energy values ranging from 20 eV to 30 keV. The energy steps are presented in Table II.

Figure 2 compares the total cross section for silicon using interpolated Mott (discrete point) and Rutherford (continuous line) cross sections (Gauvin *et al.* 1995). The values are then stored in a binary file. The name of this file is of the type “pXY.tot,” where p is the letter code of the potential (see Table I) and XY is the chemical symbol. If the chemical symbol has only one letter, then Y has the same value of X.

To compute the polar angle of collision in a Monte Carlo program, a probability of event is determined. Thus, with a random number (R), a value of the polar angle (θ) can be determined using Eq. (2).

$$R = \frac{\int_0^\theta \frac{d\sigma}{d\Omega} \sin \theta d\theta}{\int_0^\pi \frac{d\sigma}{d\Omega} \sin \theta d\theta} \quad (2)$$

For the Mott cross section, this equation does not have any analytical solution. Tabulation of the computed ratio as a function of the polar angle (θ) is needed. Figure 3 shows the values of the ratio (R) as a function of the scattering angle for silicon at 0.1 keV. This tabulation was computed for the 26

different energies. In a Monte Carlo program, a random number (R) is selected from which the proper polar angle of collision must be determined. To speed up the computation, the polar angle as a function of the ratio (R) is stored. The binary data file uses the same name of the total cross section except that the file extension is now “.r.”

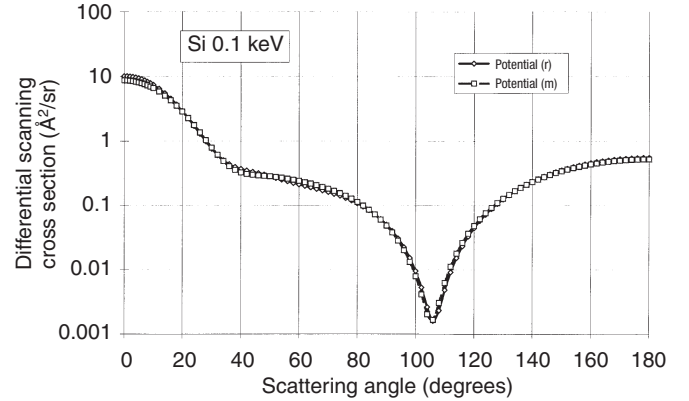


FIG. 1 Differential Mott cross section of silicon at 0.1 keV as function of the polar angle for two types of atomic potential.

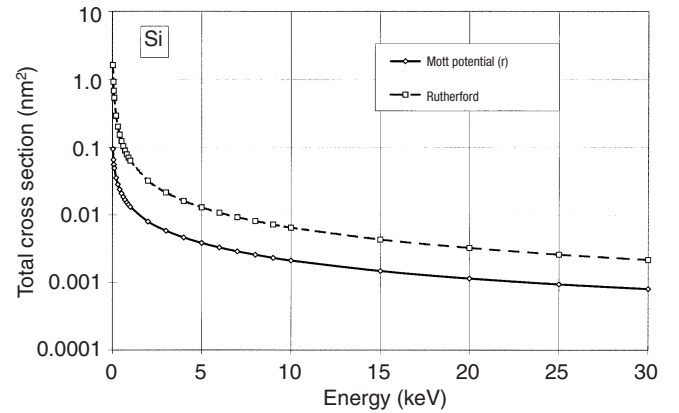


FIG. 2 Total cross section of silicon as a function of energy computed with the Rutherford and Mott theories.

TABLE I Explanation of the two types of data set available in the program for the atomic potential

Choice of the atomic potential in function of the atomic number Z	Letter code
Z < 55 TFD Z ≥ 54 HF	“m”
Z < 92 HFR	“r”

TABLE II The 26 energies at which the Mott cross section has been computed in the work of Czyzewski *et al.* (1990)

Energy (keV)	Number of Points
0.02	1
0.05	1
0.075	1
0.1 to 1 by step of 0.1 keV	10
2 to 10 by step of 1 keV	9
15 to 30 by step of 5 keV	4
Total	26

3. Programming the Mott Cross Section in a Monte Carlo Program

In this section, a brief description of the routines used to calculate the total cross section and the polar angle collision is presented. Those routines are used in the program CASINO and are also available as a stand-alone program at the following web site “<http://www.gme.usherb.ca/casino>.”

3.1 Computation of Total Cross Section

The routine “Sec_Total_MottFich” is used to compute the total cross section from a binary data file. The data file contains sequentially the values of the total cross in $\text{\AA}^2$ for 26 different energies and the values are coded on 4-bytes float. The flowchart of the routine is shown in Figure 4 and the corresponding C code in Figure 5. Four constants have to be defined

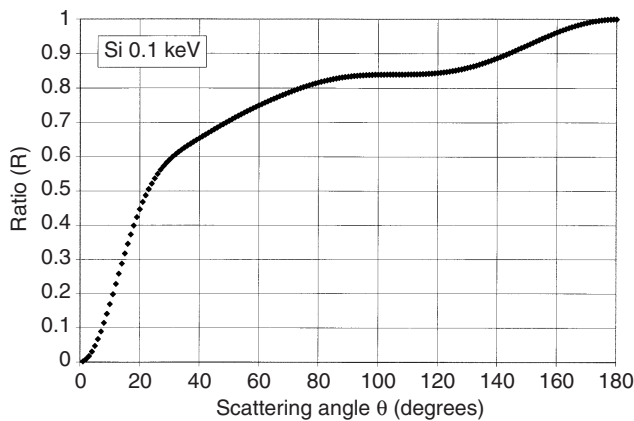


FIG. 3 Probability of scattering angle computed using Eq. (2) with Mott cross section for silicon at 0.1 keV.

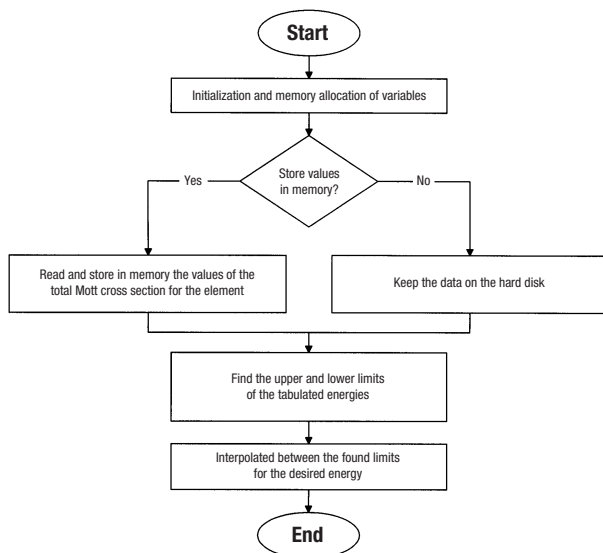


FIG. 4 Flowchart of the routine (Sec_Total_MottFich) used to compute the total cross section from tabulated values of Mott cross section.

before compiling the routine. The first one, “SYSTEM,” refers to the type of the system used; the possible values are “UNIX” or “DOS.” If the program is running on a UNIX system, the byte read has to be flipped because of the difference in the writing standard. The constant “DATALOCATION” can be set to “MEMORY” or “HARDDISK.” To achieve better performance, it is highly recommended to store an array into memory. This array will take about 100 bytes of memory for each element. If the user does not have enough memory on his computer, he will prefer to keep the data on the hard disk drive. The third constant “POTEN” refers to the atomic potential and can be set to any valid potential (see Table I). The last constant “CONVF” is a conversion factor used to convert the total cross section from $\text{\AA}^2$ to other preferred units.

To call the routine the user must specify the atomic number (Z) of the element and the energy (E) in keV, at which the total cross section must be computed. The first step is to find the name of the element corresponding to the atomic number. Then, if the program is set to use memory, the first time the routine is called the data file will be read and stored in an array. Using the “FindInd” routine, the program searches, for a given energy (E), the nearest lower (Emin) and upper (Emax) energy in the 26 tabulated energies. The total cross section at energy (E), $\sigma_t(E)$, is then calculated by interpolating between the two values found in the data file:

$$\sigma_t(E) = \sigma_t(E_{min}) + \frac{(E - E_{min}) \times (\sigma_t(E_{max}) - \sigma_t(E_{min}))}{E_{max} - E_{min}} \quad (3)$$

The return value is simply the total cross section multiplied by the conversion factor (CONVF).

3.2 Computation of Polar Angle of Collision

Given an atomic number (Z), the routine “Sec_Partiel_MottFich” computes an energy (E) and a random number (R), the polar angle of collision. The corresponding flow chart is shown in Figure 6, and Figure 7 presents the code in C language. The same constants described in section 3.1 are used except that the last one, “CONVF,” converts the angle from degree to radian. The memory management uses the same procedure described for the total cross section. The difference is that the memory needed for each element is about 20 kbytes. The binary file contains sequentially the 26 different energies. For each energy, 200 values of polar angle are stored using 4-bytes float. To optimize the accuracy of the interpolated angle, variable step has been used in the binary file. Table III shows the number of points in the function of ratio range.

A large number of points are used in the small range 0.98 to 1, since this region covers a wide angular range (c.f. Fig. 8). Even if this event occurs only 2% of the time, a small error on the R value would generate a large polar angle error and thus greatly affect the results.

With the given random number (R), the two nearest values of the tabulated ratio are found (see Table III). The routine “FindInd” is used again to determine the lower (Emin) and

Fig. 5 Routine Sec_Total_MottFich code in C language.

```

/*******/
/*
/* Calculation of total Mott Cross Section using tabulated values */
/* The input are the atomic number (int Z) and the energy in keV at which */
/* to compute the total cross section. */
/* The return values are in nm^2 */
/* If the energy is greater than 30 keV the return value is -1 */
/*
/*******/

#define SYSTEM DOS /* Possible values are DOS or UNIX.If set to UNIX */
/* the binary float are read in reverse order. */
#define LOCATION MEMORY /* Possible values are MEMORY or HARDDISK. */
/* If set to MEMORY the data are load in RAM. */
#define POTEN 'r' /* Possible values are "r" or "m". This letter */
/* refer to the potentiel used for Mott cross section */
#define CONV 1e-2 /* Conversion factor used to convert the total */
/* mott cross section from square amstrong */
#define PATH "0" /* Location of the file on the hard disk */
#define NBELMAX 15 /* Maximum number of element in same simulation */

#define DOS 0
#define UNIX 1
#define MEMORY 0
#define HARDDISK 1

#if SYSTEM == DOS
#include <alloc.h>
#endif
#include <stdio.h>
#include "chemsyb.h"

double Sec_Total_MottFich(int Z,double En)
{
    double Emin,Emax,MottExt,MottMin,MottMax,Ebid;
    int Pmin,Pmax,i,j,LongPath;
    static double **MottTot;
    static int FFichLue[94],NbEI;
    char *NomFichier;
    FILE *DataMottTot;

    void FindInd(double En,int *Pmin, int *Pmax);
    int freadUDD(double * __ptr,int type,int n,FILE *File,int SYS);

    if (En >= 30.) return(-1);

    /* Find the lower and upper position in the array of energy En */
    FindInd(En,&Pmin,&Pmax);

    /* Open the data file of the element Z */
    if(!FFichLue[Z-1] || LOCATION == HARDDISK){
        NomFichier=(char *)malloc(60*sizeof(char));
        if (!NomFichier) {
            printf("Not Enough Memory for NomFichierMott");
            exit(1);
        }
        NomFichier[0]='\0';
        strcat(NomFichier,PATH);
        LongPath=strlen(NomFichier);
        NomFichier[LongPath]=POTEN;
        NomFichier[LongPath+1]=ChemSymb[Z-1][0];
        NomFichier[LongPath+2]=ChemSymb[Z-1][1];
        NomFichier[LongPath+3]='\0';
        strcat(NomFichier,".tot");
        if ( (DataMottTot=fopen(NomFichier,"rb")) == NULL ) {
            printf("\n File %s not found ",NomFichier);
            exit(1);
        }
        free(NomFichier);
    }

    /* If it is the first call with the corresponding Z allocate the memory */
    if(!FFichLue[Z-1]){
        if(LOCATION == MEMORY){
            if (MottTot==NULL) {
                MottTot=(double **) malloc(NBELMAX*sizeof(double));
                if (!MottTot) {
                    printf("Not Enough Memory for MottTot");
                    exit(1);
                }
                NbEI=0;
            }
            MottTot[NbEI]=(double *) malloc(26*sizeof(double));
            if (!MottTot[NbEI]) {
                printf("Not enough memory for MottTot[%d]",NbEI);
                exit(1);
            }
            for (i=0;i<26;i++) {
                fread(&Ebid,sizeof(double),1,DataMottTot);
                freadUDD(&MottTot[NbEI][i],sizeof(double),1,DataMottTot,SYSTEM);
            }
            NbEI++;
            FFichLue[Z-1]=NbEI;
        }
    }

    /* Read the data from the corresponding data file */
    if(LOCATION == MEMORY){
        MottMin=MottTot[FFichLue[Z-1]-1][Pmin];
        MottMax=MottTot[FFichLue[Z-1]-1][Pmax];
    }else{
        fseek(DataMottTot,Pmin*16+8,SEEK_SET);
        freadUDD(&MottMin,sizeof(double),1,DataMottTot,SYSTEM);
        fseek(DataMottTot,Pmax*16+8,SEEK_SET);
        freadUDD(&MottMax,sizeof(double),1,DataMottTot,SYSTEM);
    }

    /* Interpolated between the two values for the energy En */
    if (Pmin == Pmax && En <= 30.)
        MottExt=MottMin;
    else {
        Emin=En[Pmin];
        Emax=En[Pmax];
        MottExt=MottMin+((MottMax-MottMin)*(En-Emin)/(Emax-Emin));
    }
    fclose(DataMottTot);
    return(MottExt*CONV);
}

#undef SYSTEM
#undef LOCATION
#undef POTEN
#undef CONV
#undef PATH

```

upper ($E_{\max}$) energy. For each energy limit, a polar angle is computed by interpolation between the tabulated values of the ratio.

$$\theta_{E_{\min}} = \theta_{E_{\min}}(R_{\min}) + \frac{(R - R_{\min}) \times (\theta_{E_{\min}}(R_{\max}) - \theta_{E_{\min}}(R_{\min}))}{R_{\max} - R_{\min}} \quad (4)$$

$$\theta_{E_{\max}} = \theta_{E_{\max}}(R_{\min}) + \frac{(R - R_{\min}) \times (\theta_{E_{\max}}(R_{\max}) - \theta_{E_{\max}}(R_{\min}))}{R_{\max} - R_{\min}} \quad (5)$$

The polar angle for energy “E” (θ_E) is computed by interpolating between “Emin” and “Emax” using Eq. (6).

$$\theta_E = \theta_{E_{\min}} + \frac{(E - E_{\min}) \times (\theta_{E_{\max}} - \theta_{E_{\min}})}{E_{\max} - E_{\min}} \quad (6)$$

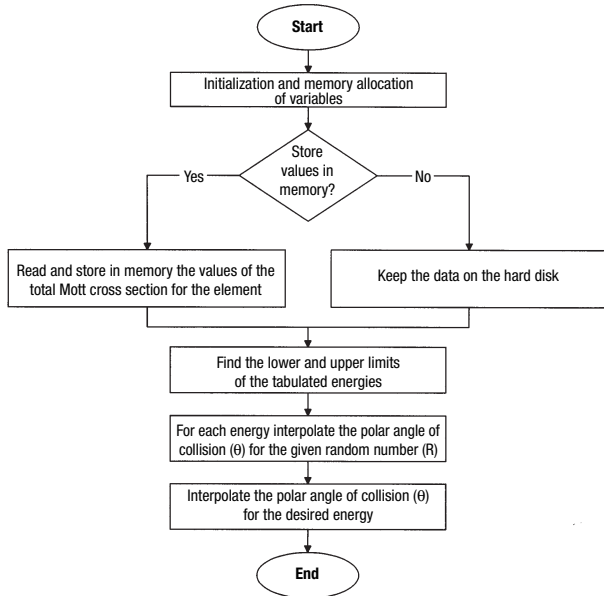


FIG. 6 Flowchart of the routine (Sec_Partial_MottFich) used to compute the polar angle of collision from tabulated values of Mott cross section.

TABLE III Description of the discrete point used to represent the polar angle of collision in function of the random number

Ratio range	Number of points
0 to 0.9 by step of 0.009	100
0.9 to 0.98 by step of 0.0016	50
0.98 to 1 by step of 0.0004	50

The computed polar angle of collision is in degrees. The computed value is then multiplied by the conversion factor (CONVF).

Results

Figure 9 shows the backscattering coefficient as a function of energy for C, Al, Ag, and Au. The backscattering coefficient was computed using the CASINO Monte Carlo program with different cross sections. The stopping power proposed by Joy and Luo (1989) was used in these simulations. All the details of the program can be found in Hovington *et al.* (1997b). Four models of cross section are shown (Browning *et al.* 1994, Drouin *et al.* 1994, Gauvin and Drouin 1993, Rutherford 1911, and the present work). The three last models are based on the Mott theory. It has been demonstrated that the Mott theory is more adequate for low-energy work (Reimer and Stelter 1986). Hence, the more accurate fit to Mott cross section is simply interpolation from tabulated values. Figure 9 also shows measured values of the backscattering coefficient performed by several authors (Joy 1995). For a high atomic number, the empirical form proposed by Browning *et al.* (1994) gives good agreement with the backscattering coefficient computed with the tabulated cross section, but for a low atomic number the agreement is less satisfactory, as shown in Figure 9(a) for carbon. This result is not surprising since, in the procedure used by Browning *et al.* (1994), the cross section was adjusted to fit the backscattering coefficient computed with the interpolated Mott cross section. This implies that the physics used in the Monte Carlo program cannot be changed. For example, the more accurate stopping power proposed by Hovington *et al.* (1997a) cannot be used because the model of Joy and Luo (1989) was used to produce the empirical equations. Moreover, the backscattering coefficient is not a sufficient criterion to determine the accuracy of the analytical fit. In Figure 10, the distribution of backscattering coefficient as a function of energy loss is shown for Al and Au. The difference between analytical and interpolation is quite significant.

The principal arguments to use the empirical form are the ease of use and the short computing time. The two routines presented in this work can easily be incorporated in any existing Monte Carlo code. The computation time is an important factor in a Monte Carlo program, since the procedure must be repeated a great many times to produce precise results. To show the performance of our routines, a speed test was performed. This test consisted of simulating 50,000 trajectories in silicon at 1 and 10 keV on a RISC 6000 model 370. The CASINO program was used with different types of cross sections. To show the difference of each model, the time to perform each simulation was normalized by the time of the Rutherford model (7min 56 s) (Table IV).

The best performance was obtained using the tabulated cross section which is 13% faster than the Rutherford model and 60% faster than the Browning *et al.* (1994) equations. In the article by Browning *et al.* (1994), an approximation was

FIG. 7 Routine Sec_Partiel_MottFich code in C language.

```

/*****
/*
/* Calculation of the polar angle of collision using the partial Mott Cross */
/* Section. The angle are tabulated and stored in data file */
/* The input are the atomic number (int Z), the energy (En) in keV at which */
/* to compute the total cross section and a random number (Ran) uniformly */
/* distributed between 0 and 1. */
/* The return values are in radian */
/* If the energy is greater than 30 keV the return value is -1 */
/*
*****/

#define SYSTEM DOS /* Possible values are DOS or UNIX.If set to UNIX */
/* the binary float are read in reverse order. */
#define LOCATION MEMORY /* Possible values are MEMORY or HARDDISK. */
/* If set to MEMORY the data are load in RAM. */
#define POTEN 'r' /* Possible values are "r" or "m". This letter */
/* refer to the potentiel used for Mott cross section */
#define CONVF 1.7453292e-2 /* Conversion factor used to convert the polar */
/* angle of collision from degre to radian */
#define PATH "0" /* Location of the file on the hard disk */

double Sec_Partiel_MottFich(int Z,double En,double Ran)
{
    double Emin,Emax,*Ebid,ThetaEmin,ThetaEmax,ThetaExt,Divi,R1;
    float ThetaEmin1,ThetaEmin2,ThetaEmax1,ThetaEmax2;
    int Pmin,Pmax,i,j,LongPath,Indice,NbEl;
    static float **MottPar;
    static int FFichLue[94];
    char *NomFichier;
    FILE *DataMottPar;

    void FindInd(double En,int *Pmin, int *Pmax);
    int freadUDF(float * __ptr,int type,int n,FILE *File,int SYS);

    if (En >= 30.) return(-1);
    if (Ran >= 0.99999999) return(3.14159265);
    if (Ran <= 0.00000001) return(0.);

    /* Find the lower and upper position in the array of energy En */
    FindInd(En,&Pmin,&Pmax);

    /* Open the data file of the element Z */
    if(!FFichLue[Z-1] || LOCATION == HARDDISK){
        NomFichier=(char *)malloc(60*sizeof(char));
        if (!NomFichier) {
            printf("Not Enough Memory for NomFichierMott");
            exit(1);
        }
        NomFichier[0]='\x0';
        strcat(NomFichier,PATH);
        LongPath=strlen(NomFichier);
        NomFichier[LongPath]=POTEN;
        NomFichier[LongPath+1]=ChemSymb[Z-1][0];
        NomFichier[LongPath+2]=ChemSymb[Z-1][1];
        NomFichier[LongPath+3]='\x0';
        strcat(NomFichier,".r");
        if ((DataMottPar=fopen(NomFichier,"rb")) == NULL) {
            printf("\n File %s not found ",NomFichier);
            exit(1);
        }
        free(NomFichier);
    }

    /* If it is the first call with the corresponding Z allocate the memory */
    if(!FFichLue[Z-1]){
        if(LOCATION == MEMORY){
            if (MottPar==NULL) {
                MottPar=(float **) malloc(NBELMAX*sizeof(double));
                if (!MottPar) {
                    printf("Not Enough Memory for MottPar");
                    exit(1);
                }
                NbEl=0;
            }
            MottPar[NbEl]=(float *) malloc(5200*sizeof(float));
            /* 5200=26(Energie)*200(angle) */
            if (!MottPar[NbEl]) {
                printf("Not enough memory for MottPar[%d]",NbEl);
                exit(1);
            }
        }
    }
}

```

(continued)

FIG. 7 Routine Sec_Partial_MottFich code in C language. (continued)

```

        for (i=0;i<26;i++) {
            for(j=0;j<200;j++){
freadUDF(&MottPar[NbEl][j]+(*200),sizeof(float),1,\
                                                DataMottPar,SYSTEM);
            }
            NbEl++;
            FFichLue[Z-1]=NbEl;
        }
    }

/* Find the position in the array of Ran with the variable step */
if(Ran <=0.9){
    Indice=Ran/0.009;
    Divi=0.009;
    R1=Indice*0.009;
}else if(Ran>0.9 && Ran <= 0.98){
    Indice=(Ran-0.9)/0.0016+100;
    Divi=0.0016;
    R1=0.9+0.0016*(Indice-100);
}else{
    Indice=(Ran-0.98)/0.0004+150;
    Divi=0.0004;
    R1=0.98+0.0004*(Indice-150);
}

/* Read the data from the corresponding data file */
if(LOCATION == MEMORY){
    ThetaEmin1=MottPar[FFichLue[Z-1]-1][Pmin*200+Indice-1];
    ThetaEmin2=MottPar[FFichLue[Z-1]-1][Pmin*200+Indice];
    ThetaEmax1=MottPar[FFichLue[Z-1]-1][Pmax*200+Indice-1];
    ThetaEmax2=MottPar[FFichLue[Z-1]-1][Pmax*200+Indice];
}else{
    fseek(DataMottPar,(Pmin*200+Indice-1)*4,SEEK_SET);
    freadUDF(&ThetaEmin1,sizeof(float),1,DataMottPar,SYSTEM);
    fseek(DataMottPar,(Pmin*200+Indice)*4,SEEK_SET);
    freadUDF(&ThetaEmin2,sizeof(float),1,DataMottPar,SYSTEM);
    fseek(DataMottPar,(Pmax*200+Indice-1)*4,SEEK_SET);
    freadUDF(&ThetaEmax1,sizeof(float),1,DataMottPar,SYSTEM);
    fseek(DataMottPar,(Pmax*200+Indice)*4,SEEK_SET);
    freadUDF(&ThetaEmax2,sizeof(float),1,DataMottPar,SYSTEM);
}

/* If Ran is less than 0.009 the value the polar angle is 0 */
if(Indice == 0){
    ThetaEmin1=0;
    ThetaEmax1=0;
}

/* Interpolated for each energy the polar angle for the random number Ran */
ThetaEmin=(ThetaEmin2-ThetaEmin1)*(Ran-R1)/Divi+ThetaEmin1;
ThetaEmax=(ThetaEmax2-ThetaEmax1)*(Ran-R1)/Divi+ThetaEmax1;

if(En < 0.02){ /* Extrapolated between 0 ev and 20 eV */
    ThetaEmin1=0;
    ThetaEmin2=180;
}

/* Interpolated between the two polar angle for the energy En */
if (Pmin == Pmax && En <= 30.)
    ThetaExt=ThetaEmin;
else {
    Emin=EnT[Pmin];
    Emax=EnT[Pmax];
    ThetaExt=ThetaEmin+((ThetaEmax-ThetaEmin)*((En-Emin)/(Emax-
Emin)));
}
if (ThetaExt < 0. || ThetaExt > 180){
    printf ("\n Problem in routine Sec_Eff_PartFich: ThetaExt < 0 or\
ThetaExt > 180!! ");
    exit(1);
}
fclose(DataMottPar);
return(ThetaExt*CONVF);
}
#undef SYSTEM
#undef LOCATION
#undef POTEN
#undef CONVF
#undef PATH
}

/* Read a double in the File *File and convert if the system is UNIX */
int freadUDD(double * __ptr,int type,int n,FILE *File,int SYS)
{
    int i,ret;
    double *tmp;
    char temp[16];

    if(!SYS)
        ret=fread(__ptr,type,n,File);
    else{
        for(i=type-1;i>=0;i--){
            ret=fread(&temp[i],sizeof(char),n,File);
        }
        tmp=&temp;
        * __ptr=tmp;
    }
    return(ret);
}

/* Read a float in the File *File and convert if the system is UNIX */
int freadUDF(float * __ptr,int type,int n,FILE *File,int SYS)
{
    int i,ret;
    float *tmp;
    char temp[16];

    if(!SYS)
        ret=fread(__ptr,type,n,File);
    else{
        for(i=type-1;i>=0;i--){
            ret=fread(&temp[i],sizeof(char),n,File);
        }
        tmp=&temp;
        * __ptr=tmp;
    }
    return(ret);
}

void FindInd (double En, int *Pmin, int *Pmax)
{
    int i;

    if (En < EnT[0]) {
        *Pmin=0;
        *Pmax=0;
        return;
    }
    if (En > EnT[25]) {
        *Pmin=25;
        *Pmax=25;
        return;
    }

    if (En <= 0.1) {
        i=-1;
        do
            i++;
        while ( EnT[i] < En);
        if ( En == EnT[i]) {
            *Pmin=i;
            *Pmax=i;
        }
        else {
            *Pmin=i-1;
            *Pmax=i;
        }
    }
    else if ( En > 0.1 && En <=1) {
        *Pmin=(int)(En*10)+2;
        *Pmax=*Pmin+1;
        return;
    }

    else if ( En >1 && En <=10) {
        *Pmax=(int)(En)+12;
        *Pmin=*Pmax-1;
        return;
    }
    else if (En > 10) {
        *Pmax=(int)((En-10)/5)+22;
        *Pmin=*Pmax-1;
        return;
    }
}

```

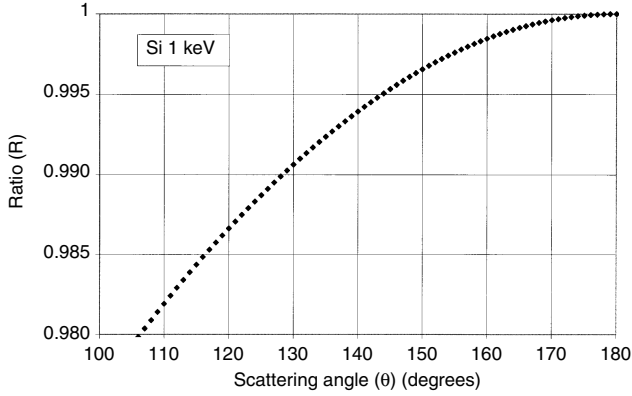



FIG. 8 Large variation of the polar of collision for small range of probability for silicon at 1 keV.

proposed to decrease the computation time. Even with this approximation, the tabulated Mott cross section is still 38% faster. Clearly, there are no more reasons to use the empirical form of the Mott cross section, since tabulated cross sections are faster, more accurate, and can easily be added to any Monte Carlo code.

Conclusion

The latest Monte Carlo program uses the theory of Mott to compute the total cross section and the polar angle of collision. The Rutherford theory is not appropriate for low-energy applications. In the present work, two types of data sets have been presented to calculate the Mott cross section with different atomic potentials. These data sets are stored in binary files and use about 1 Mbyte of disk space for the 94 first elements of the Periodic Table. Two routines have been presented to compute the total cross section and the polar angle of collision from tabulated values. By definition, the interpolated Mott cross section is more accurate than any empirical formula. In addition, it has been shown that the routines presented in this work are faster than any analytical elastic cross section available at the present time.

TABLE IV Comparison of the performance of different types of cross section. The time of each model has been normalized by the time of the Rutherford model. 50,000 trajectories have been simulated in silicon at 1 and 10 keV. The simulations were performed on RISC 6000 model 370

Sample test	Si 1 keV (%)	Si 10 keV (%)
Rutherford (1911)	100	100
Gauvin and Drouin (1993)	86.7	131
Browning <i>et al.</i> (1994)	83.2	140.8
Browning <i>et al.</i> (1994) approximation	71.6	123.4
This work	51.8	91.6

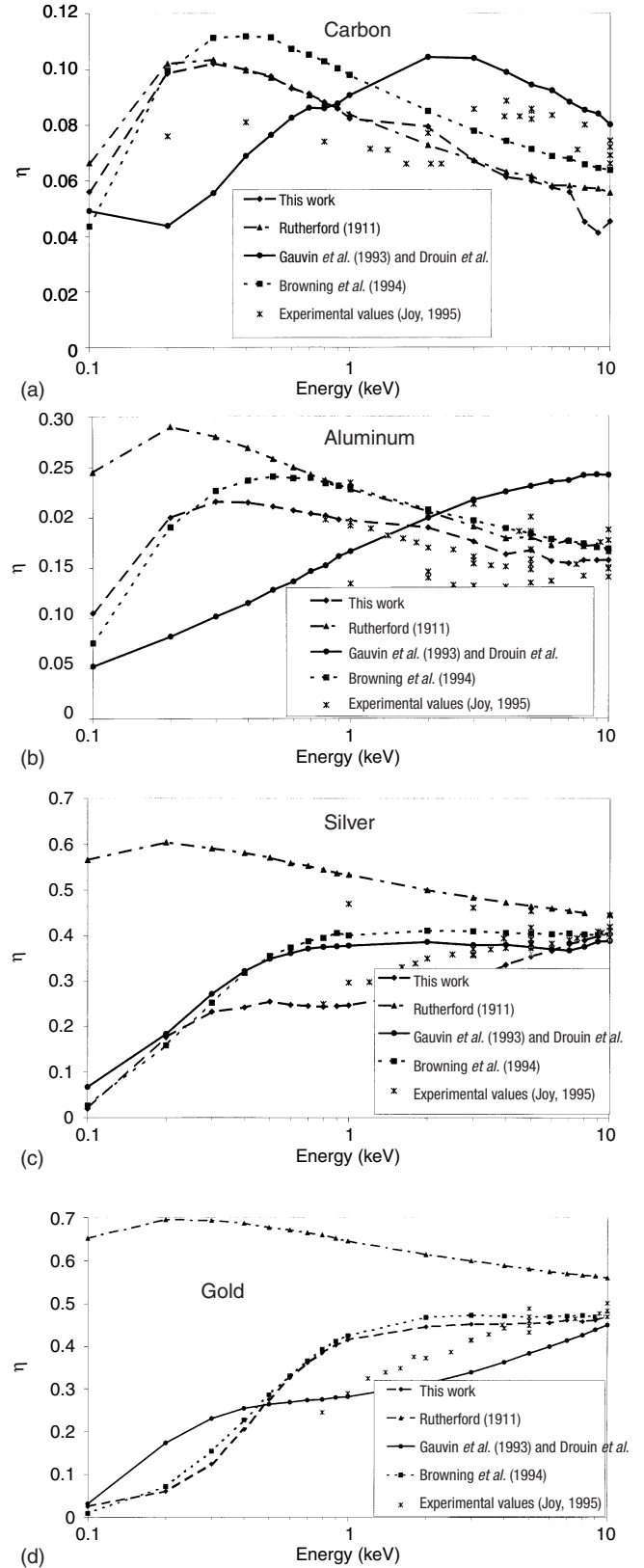
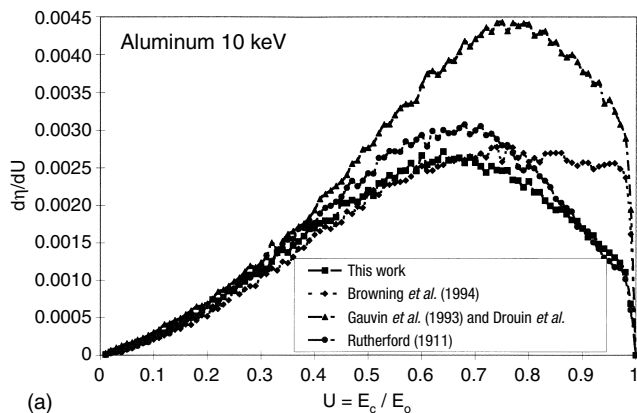
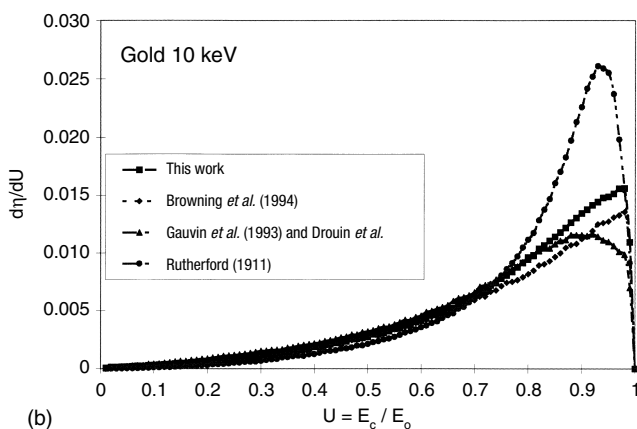


FIG. 9 Backscattering coefficient as a function of energy computed using different model of cross section for: (a) carbon, (b) aluminum, (c) silver, and (d) gold.



(a)



(b)

FIG. 10 Energy distribution of backscattered electron for (a) aluminum and (b) gold at 10 keV beam energy.

References

- Bethe HA: *Handbook of Physics*, Vol. 24. Springer, Berlin (1933) 273
- Bonham RA, Strand TG: Analytical expression for potential of neutral Thomas-Fermi-Dirac atom and for the corresponding atomic scattering factors for x-rays and electrons. *J Chem Phys* 39, 2200–2204 (1963)
- Browning R, Li TZ, Chui B, Ye J, Pease RFW, Czyzewski Z, Joy DC: Empirical form for the electron/atom elastic scattering cross sections from 0.1 to 30 keV. *J Appl Phys* 76, 4, 2016–2022 (1994)
- Cox HL, Bonham RA: Elastic electron scattering amplitude for neutral atoms calculated using the partial wave method at 10, 40, 70, and 100 keV for $z=1$ to $z=54$. *J Chem Phys* 47, 2599–2608 (1967)
- Czyzewski Z, MacCallum DO, Romig A, Joy DC: Calculations of Mott scattering cross section. *J Appl Phys* 68, 7, 3066–3072 (1990)
- Drouin D, Gauvin R, Joy DC: Computation of polar angle of collisions from partial elastic Mott cross-sections. *Scanning* 16, 67–77 (1994)
- Gauvin R, Drouin D: A formula to compute total elastic Mott cross-sections. *Scanning* 15, 140–150 (1993)
- Gauvin R, Drouin D, Hovington P: The use of Monte Carlo simulations in scanning electron microscopy (SEM) to understand contrast mechanisms of SEM images of bi-phased materials at low voltages. *Scanning* 17(suppl. V), V-12 (1995)
- Hovington P, Drouin D, Gauvin R: CASINO: A new era of Monte Carlo code in C language for the electron beam interaction—Part III: Stopping power. *Scanning* 19, 29–35 (1997a).
- Hovington P, Drouin D, Gauvin R: CASINO: A new era of Monte Carlo code in C language for the electron beam interaction—Part I: Description of the program. *Scanning* 19, 1–14 (1997b)
- Joy DC, Luo S: An empirical stopping power relationship for low-energy electrons. *Scanning* 11, 176–180 (1989)
- Joy DC: A database on electron-solid interactions. *Scanning* 17, 270–275 (1995)
- Mott NF, Massey HSW: *The Theory of Atomic Collisions*. Clarendon Press, Oxford (1949) 243–244
- Reimer L, Stelter D: FORTRAN 77 Monte Carlo program for minicomputers using Mott cross sections. *Scanning* 8, 265–277 (1986)
- Rutherford E: Elastic cross section. *Phil Mag* 21, 669–680 (1911)
- Strand TG, Bonham RA: Analytical expression for the Hartree-Fock potential of neutral atom and for the corresponding atomic scattering factors for x-rays and electrons. *J Chem Phys* 40, 1686–1691 (1964)